

Speedup Javascript engine

Since Firefox Add-Ons run in the same engine as the web page content, they too benefit from speedups in Firefox's JavaScript engine

Better Thunderbird

Thunderbird 3.3 will use the same base as Firefox 4 bringing some of its speed and features to Mozilla Messaging's email client

Developer corner

create for your extension will thus be compatible with other frameworks.

A file called "main.js" inside the "lib" directory will be where most of your code will lie. It should already have been created by the "cfx init" command you used earlier, and will contain some basic code. You can remove this code to get a clean slate.

Since we wish to modify the context menu for Firefox, we need to import the JAtpack Core module that offers that functionality. Add the following line to your 'main.js' file:

```
var contextMenu =
require("context-menu");
```

This will let us use the context menu features defined in the "context-menu" module by referencing them through the contextMenu variable.

We will also be using the tabs API, so include that with:

```
var tabs = require("tabs");
```

Now let us create a context menu item. The following is the complete code that we will explain a little later:

```
var postTwitter = contextMenu.
Item({
  label: "Tweet",
  context: contextMenu.SelectionContext(),
  contentScript: 'on("click",
function (node, data) {' +
    ' var selection = document.getSelection().
toString();'+
    '
postMessage(selection);' +
    '});',
  onMessage: function (selectedText) {
    tabs.open("http://
twitter.com/?status="+encodeURIComponent(selectedText));
  }
});
```

We are creating a new context menu item with a label of "Tweet", and three other parameters called context, contentScript, onMessage which will explain next.

How the code works

The context menu, quite obviously, is context-sensitive. What this means is, that the options which appear on the

context menu will vary depending on the item you right-click on. We need to specify a context for our menu item to appear in, and Firefox will automatically manage which item to display and when.

This context can be specified in multiple ways:

- **contextMenu.SelectorContext(selector):** The context menu will be displayed on all elements that match the specified CSS selector.
- **contextMenu.SelectionContext():** The context menu will be displayed if there is a selection.
- **contextMenu.URLContext(matchPattern):** The context menu will be displayed if the URL of the page matches the specified pattern
- **contextMenu.PageContext():** Displayed everywhere on the page.

You can even specify an array of contexts, in which case the menu will be displayed if all contexts match. You can even have a custom context, where you code the function that can perform any number of checks before displaying the item in the menu.

An explanation of what is going on here is in order.

The contentScript is the part of the code that is handled by the context menu. This code is kept separate from the main code of the add-on and is specified in a string – although you could load it from a file.

The reason for keeping the code that is executed by the context menu separate from the rest of the add-on code is to make it simpler in the future to move to a model where the add-on, the browser UI, and the content are all running in different processes. This is similar to how Google Chrome works, where each tab is a different process, each add-on is a different process and the browser itself has its own process. Firefox too will embrace this model and when it does add-ons that are developed using the Add-On SDK will continue working.

Since the code running in the context menu is separate from the rest, it cannot directly access variables from the main add-on so how do we tell our add-on that it has been clicked and see a reaction? We use the postMessage function that can be used to send strings and objects (with some

restrictions) from the context menu to the add-on. In our case, we are posting the text selected in the document using document.getSelection().toString().

On the other end, our add-on eagerly awaits a message from the context menu with the onMessage function. This function receives the message we sent from the context menu and can use it to perform any action.

When it receives the message, in this case it opens a new tab with the URL:

```
"http://twitter.com/?status="+encodeURIComponent(selectedText)
```

The encodeURIComponent function converts our selection to a form that can be safely passed from a URL. This status text is then appended to the URL "http://twitter.com/?status=" that can be used to specify a tweet message while opening the Twitter home page.

You will need to be already logged into Twitter for this. If you are logged in this will open the Twitter home page with the selected text already entered in the tweet box so you can go ahead and click the "Tweet" button to post it.

You can now run "cfx run" from the command line to launch this extension in your browser. When you run the "cfx run" command for the first time, it will simply modify your 'package.json' file to include a unique id; you will have to run it a second time to launch it in your browser.

Note that by default the "cfx" command might launch the the wrong version of Firefox (Firefox 3.6) which will fail to enable this extension. In this case you should specify which Firefox version to use by providing its path in the commandline:

```
cfx run -b C:\path\to\firefox4\
firefox.exe
OR crx run --binary=C:\path\
to\firefox4\firefox.exe
```

You can see how simple developing extensions for Firefox is about to become! With just a few lines of code we have a functional extension of some utility. Currently the Add-On SDK is not even complete, it is the first beta version! The best part is all extensions using this SDK will get features such as restart-less installation, future compatibility and a consistency in appearance and function without any extra effort. ■